Important Instructions

- การส่ง Assignment ทำผ่าน Github Classroom
  - งานใน A09 เป็นงานต่อจาก A08 แต่ใช้ Repository ตั้งต้นที่ให้มา ผ่านAssignment Invitation Link สำหรับ A09
  - การส่ง Assignment ภายใต้ Student Identity ที่กำหนดไว้ตั้งแต่ตอนรับ Invitation ของ A01

- นิสิตทดสอบ
  - Assignment มาพร้อม Test Script สำหรับทดสอบ project ให้นิสิตทดสอบตาม instruction ในสไลด์ชุดนี้

- นิสิตส่งงานโดย
  - Deploy เว็บไซต์ที่สร้างจาก Assignment ขึ้น Vercel
  - Commit และ push ขึ้น repository ที่ได้จาก Assignment และ Personal Repository ที่สร้างขึ้นใหม่
  - นิสิตต้องใส่ link ไปยัง Vercel website ของนิสิต ใน `README.md` ใน Repository ทั้งคู่

Prepare Class Environment

- เปิด Folder สำหรับเก็บหน้า React Web ที่สร้างไว้สำหรับ A09 ด้วย Visual Studio Code
- ไปที่ Menu `View` > `Terminal`
  - Clone Repo ของตนเอง พิมพ์คำสั่ง `git clone` *`REPO_URL_FROM_GITHUB`*

```
TERMINAL                                                    powershell
[redacted]eb> git clone https://github.com/machigar-cu/
[redacted].git
```

- เปิด Folder ที่ Clone มาได้ (`a09-swdevprac2-xxxxxx`) ใน Visual Studio Code
  - จะพบไฟล์ตั้งต้นของ project

Prepare Class Environment

- ไปที่เมนู View > Terminal
  - Configure git username และ email สำหรับ local repository

```
git config --local user.email "example@email.com"
git config --local user.name "your_userName"
```

```
[redacted]ar> git config --local user.email "[redacted]@gmail.com"
[redacted]ar> git config --local user.name "machigar"
```

  - ติดตั้ง node packages `npm install`
  - Copy โฟล์เดอร์ public และ src จาก project ของ A08 มาทับ โฟล์เดอร์ public และ src ใน project ของ A09 นี้
  - ลอง run project จะได้ project ที่ทำไว้ใน A09 `npm run dev`

Prepare Class Environment

- วางไฟล์ `.env.local` ที่ให้มาสำหรับ setup NextAuth ไว้ที่ root ของ project directory

- เนื่องจาก Vercel ต้องการ Strict Typing ในการ deploy ขึ้น Vercel ต้องนิยาม Type ของข้อมูล รวมถึงข้อมูลชนิด session ใน Next-Auth ด้วย
  - ดาวน์โหลดไฟล์ `next-auth.d.ts` ที่ให้มา
  - วางไฟล์ไว้ที่ `src\next-auth.d.ts`

```
import NextAuth from 'next-auth'

declare module "next-auth" {
    interface Session {
        user: {
            _id: string,
            name: string,
            email: string,
            role: string,
            token: string
        }
    }
}
```

Venue Explorer Backend

Assignment นี้ จะให้ทำการ POST Log-in data ไปยัง Backend เพื่อทำการ Log-in

Backend สำหรับ Assignment นี้ สร้างไว้ให้อยู่แล้ว นิสิตสามารถเข้าใช้งาน API ได้ดังนี้

- API สำหรับ Log-in
  https://a08-venue-explorer-backend.vercel.app/api/v1/auth/login
- API สำหรับดึงข้อมูล user profile
  https://a08-venue-explorer-backend.vercel.app/api/v1/auth/me

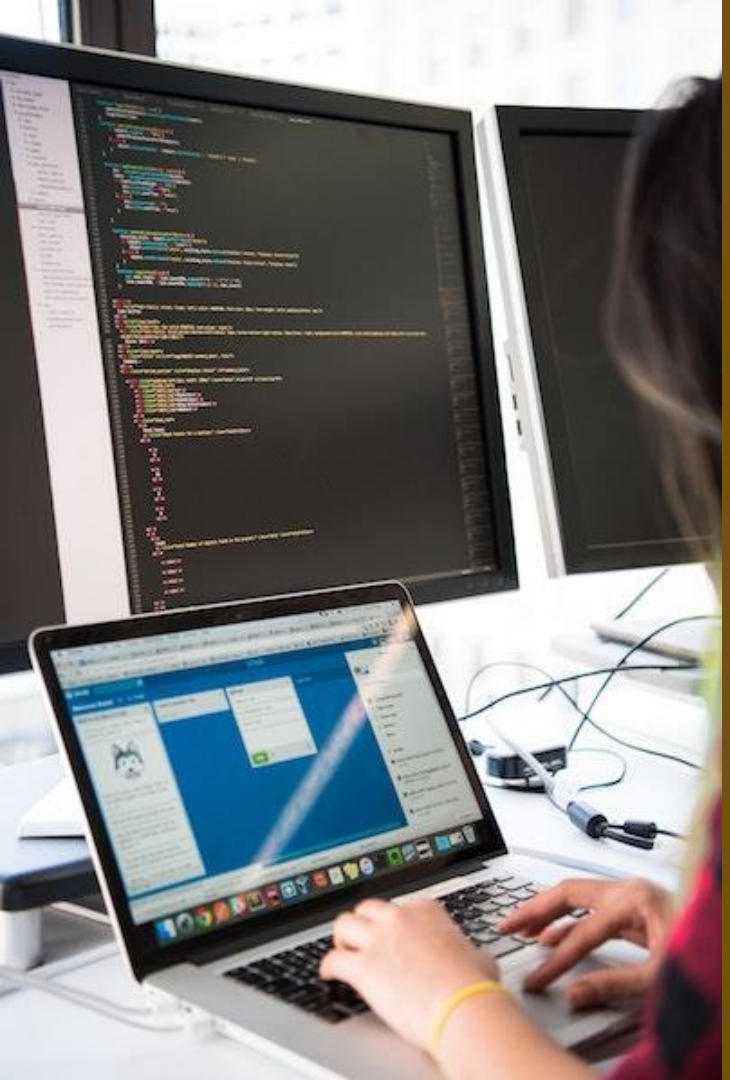
- Backend มี user ที่มี role เป็น user ธรรมดา สร้างไว้ให้เรียบร้อยแล้ว โดยมีข้อมูลดังนี้
  - name: `Alice`
  - email: `alice@eventplanner.com`
  - password: `g00dD@y$`
  - tel: `0854439954`
  - role: `user`

Venue Explorer Backend

Backend สำรอง ในกรณีที่ Backend ในสไลด์ที่ 4 โหลดเต็ม สามารถใช้ Backend ต่อไปนี้

- https://a08-venue-explorer-backend-2.vercel.app/api/v1/auth/login
  และ https://a08-venue-explorer-backend-2.vercel.app/api/v1/auth/me

- https://a08-venue-explorer-backend-3.vercel.app/api/v1/auth/login
  และ https://a08-venue-explorer-backend-3.vercel.app/api/v1/auth/me



Assignment 09:
NextAuth & Data Posting

NextAuth & Data Posting

ใน venue-explorer-app เพิ่มการ Sign-In เป็นการ Log-In ไปยัง Backend API

- เพิ่มการใช้งาน **NextAuth** โดยใช้ NextAuth เข้าไปใน venue-explorer-app

- ในโฟลเดอร์ src\libs ที่เก็บฟังก์ชั่นต่าง ๆ
  - สร้างไฟล์ userLogIn.tsx บรรจุ async ฟังก์ชั่น userLogIn สำหรับส่ง POST Request ไปยัง Backend API ที่ https://*backend-domain*/api/v1/auth/login
    - Content-Type เป็น application/json
    - body เป็น JSON ในรูปแบบดังต่อไปนี้

      userEmail = ค่าอีเมล์ที่ user ป้อน

      userPassword = รหัสผ่านของ user

```
{
        email: userEmail,
        password: userPassword,
}
```

- ให้การ Sign-In ผ่าน NextAuth เป็นการเรียกฟังก์ชั่น userLogIn นี้

NextAuth & Data Posting

- ใน component TopMenu สร้าง Link ไปยังหน้า Sign-In ไว้ด้านซ้ายมือ
  - รองรับการ Sign-In ด้วย email และ password

Sign-In	Menu Item **Booking**	**วาง logo**

  - เมื่อ Sign-In สำเร็จ Link นี้จะเปลี่ยนเป็น Link ไปยังหน้า Sign-Out แทน

Sending Bearer Token with API Request

- ปรับแต่ง NextAuth ให้รองรับ Token ที่ส่งมาจาก Backend API หลังการ Log-In
- ในโฟลเดอร์ `src\libs` ที่เก็บฟังก์ชั่นต่าง ๆ
  - สร้างไฟล์ `getUserProfile.tsx` บรรจุ `async` ฟังก์ชั่น `getUserProfile` สำหรับส่ง GET Request ไปยัง Backend API พร้อมแนบ Bearer Token โดยส่งไปที่ https://*backend-domain*/api/v1/auth/me
  - ฟังก์ชั่น `getUserProfile` return ค่าเป็น JSON Object ที่ได้รับมาจาก backend API
    - JSON Object ของ response จาก Backend API มีโครงสร้างดังตัวอย่าง

```
{
  "success": true,
  "data": {
    "_id": "67d2b3071e59d13be2c033a6",
    "name": "Alice",
    "email": "alice@eventplanner.com",
    "tel": "0854439954",
    "role": "user",
    "createdAt": "2025-03-13T10:27:19.226+00:00",
    "__v": 0
  }
}
```

Sending Bearer Token with API Request

- ที่หน้า Page `page.tsx` ของ route สำหรับการจองสถานที่จัดเลี้ยง ที่ http://localhost:3000/booking
  - หาก User ทำการ Log-In แล้ว ให้เรียกฟังก์ชั่น `getUserProfile` เพื่อดึงข้อมูล User Profile
  - แสดงข้อมูล User Profile ไว้ที่ด้านบนของ Page
    - Name จาก `name` ของ User
    - Email จาก `email` ของ User
    - Tel. จาก `tel` ของ User
    - Member Since จาก `createdAt` ของ User

Sending Bearer Token with API Request

- เพิ่มการเข้าถึง User Session จาก Client Component โดย
  - สร้าง client component ใน `src/ providers/NextAuthProvider.tsx` ให้ทำหน้าที่เป็น Customized Session Provider
  - ที่ `app/layout.tsx` เพิ่มการเข้าถึง Session ผ่าน `getServerSession()` แล้วนำไปใช้เป็น session สำหรับ `NextAuthProvider`
  - นำ `NextAuthProvider` ไปครอบ children ทั้งหมด เพื่อให้ client component ทั้งหมดเข้าถึง session ได้

- ที่ Component `Banner.tsx` บนหน้าแรก
  - หาก User ได้ทำการ Log-In แล้ว ให้แสดงข้อความที่ด้านบนขวาของ Banner ว่า **Welcome** *user*
    - *user* = ชื่อ (name) ของ User ที่ Log-In
      เช่น **Welcome Alice**

Sending Bearer Token with API Request

- ทดสอบด้วยข้อมูลสำหรับ log-in ของ user account ที่มีอยู่แล้วบน Backend API ดังนี้
  - email: `alice@eventplanner.com`
  - password: `g00dD@y$`

Assignment test cases

Assignment 09 มี 3 test cases (รวม auto-grade 6 คะแนน)

- ลอง run test case ที่เครื่องของตนเอง ไปที่เมนู `View > Terminal`
  - Run test case 1 (3 คะแนน) ด้วยคำสั่ง

    ```
    npm run test -- case1.test.tsx
    ```

  - Run test case 2 (1 คะแนน) ด้วยคำสั่ง

    ```
    npm run test -- case2.test.tsx
    ```

  - Run test case 3 (2 คะแนน) ด้วยคำสั่ง

    ```
    npm run test -- case3.test.tsx
    ```

Deploy to Vercel

- ในขณะ Deploy ขึ้น Vercel นั้น Code ต่าง ๆ จะได้รับการ Build โดยมีการตรวจสอบ Type Safely ต่าง ๆ ตัวอย่างเช่น
  - Strict Typing
  - Type validity รวมถึง undefined ต่าง ๆ โดยเฉพาะจาก optional props

- ทดลอง build ใน Local Environment ก่อน Deploy ขึ้น Vercel ได้ผ่านคำสั่ง

```
npm run build
```

- การ build จะทำให้มีการ pre-render หน้าต่าง ๆ ให้เป็น static content และ Optimize เพื่อลด runtime โดยทดลอง run ผลลัพธ์ที่ได้จากการ build ดังนี้

```
npm run start
```

  - จะเห็นได้ว่า website สามารถโหลดได้รวดเร็ว และมีการ optimize การทำงานเพื่อให้ delay น้อยที่สุด (สังเกตได้ว่าไม่มีการทำงานของ progress bar ที่ตั้ง delay เอาไว้)

Deploy to Vercel

- ใน NextAuth สำหรับ NextJs 13.4+ หากนิยาม `authOptions` ใน `route.ts` แบบดั้งเดิม จะไม่สามารถ build project ได้ จึงไม่สามารถ deploy ขึ้น Vercel ได้
  - แสดง `Type Error: 'AuthOptions' is not assignable to type 'never'`
- แก้ไขได้โดยแยกการประกาศ `authOptions` ไปยังไฟล์อื่นที่ไม่ใช่ `route.ts`

- สร้างไฟล์ `src\app\api\auth\[...nextauth]\authOptions.ts`
- ย้ายส่วนการประกาศ `authOptions` จาก `route.ts` มายัง `authOptions.ts`

Deploy to Vercel

- authOptions.ts

```
import { AuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";
import UserLogin from "@/libs/userLogIn";

export const authOptions: AuthOptions = {
  providers: [
    CredentialsProvider({
      // The name to display on the sign in form (e.g. "Sign in with...")
      name: "Credentials",
      // `credentials` is used to generate a form on the sign in page.
      // You can specify which fields should be submitted, by adding keys to the `credentials` object.
      // e.g. domain, username, password, 2FA token, etc.
      // You can pass any HTML attribute to the <input> tag through the object.
      credentials: {
        email: { label: "Email", type: "email", placeholder: "email" },
        password: { label: "Password", type: "password" },
      },
      async authorize(credentials, req) {
        if (!credentials) return null;
        const user = await UserLogin(credentials.email, credentials.password);
        if (user) {
          // Any object returned will be saved in `user` property of the JWT
          return user;
        } else {
          // If you return null then an error will be displayed advising the user to check their details.
          return null;

          // You can also Reject this callback with an Error thus the user will be sent to the error page with the
        }
      },
    }),
  ],
  session: { strategy: "jwt" },
  callbacks: {
    async jwt({ token, user }) {
      return { ...token, ...user };
    },
    async session({ session, token, user }) {
      session.user = token as any;
      return session;
    },
  },
};
```

Deploy to Vercel

- route.ts

src > app > api > auth > [...nextauth] > TS route.ts > ...

```
1  import NextAuth from "next-auth/next";
2  import {authOptions} from "@/app/api/auth/[...nextauth]/authOptions"
3
4  const handler = NextAuth(authOptions);
5  export { handler as GET, handler as POST };
```

- ในทุกไฟล์ที่มีการ import authOptions จาก route.ts ให้เปลี่ยนเป็นการ import จาก
  import { authOptions } from "../api/auth/[...nextauth]/authOptions";
  แก้ไขให้ import authOptions จาก authOptions.ts

```
import { authOptions } from "../api/auth/[...nextauth]/authOptions";
```

- ทดสอบการ build ด้วยคำสั่ง npm run build

Development & Deployment

- ใช้แนวทางการ Deploy แบบเดียวกับใน Assignment A03 ถึง A08
  - สร้าง Repository บน Personal GitHub Account สำหรับ Assignment A09
    - ดู Instructions ใน Assignment A01
  - Config ให้ local project มี multiple remote repositories และ Push งานขึ้น repositories ทั้งสอง
    - ดู Instructions ใน Assignment A01
  - Deploy project จาก Personal GitHub Account ขึ้น Vercel
    - ดู Instructions ใน Assignment A01
  - Configure และ Deploy บน Vercel โดย
    ระบบจะตรวจจับได้ว่าเป็น Next.js Framework

- ใส่ link ไปยัง Vercel website ของนิสิต ใน README.md ใน Repository

Add Environment Variables to Vercel

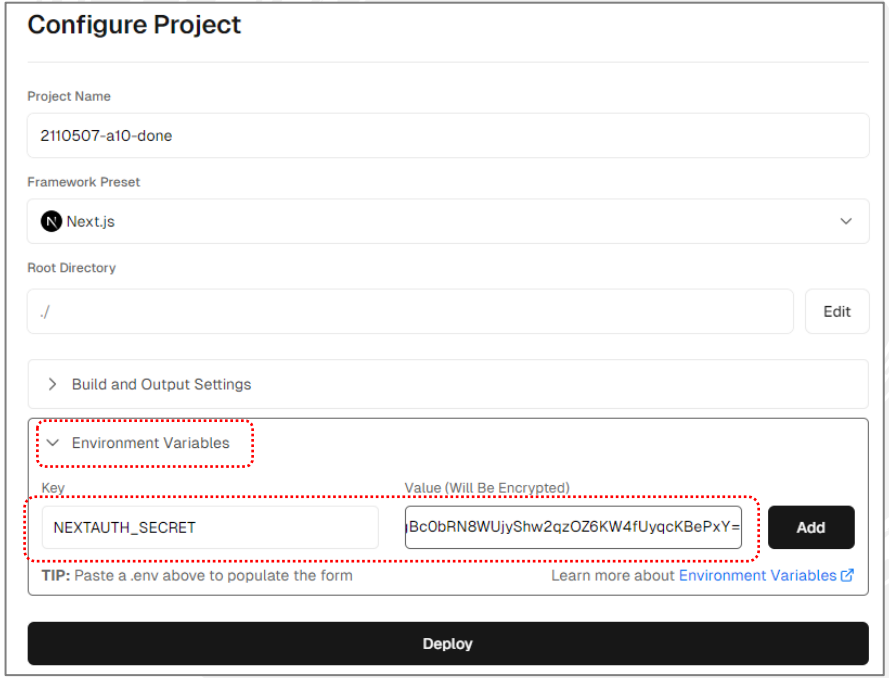
- เพื่อให้ Site ที่ Deploy ขึ้น Vercel สามารถทำงานได้แบบ Site ที่เราทดสอบที่ localhost เราต้องตั้ง environment variables ซึ่งประกาศไว้ที่ไฟล์ `.env.local` ให้กับ Vercel ด้วย
- ในการ Configure Project ช่วง deploy
  - เลือกส่วน Environment Variables

กำหนด Environment Variables ต่อไปนี้

- `NEXTAUTH_SECRET` กำหนดค่าเป็นค่าเดียวกับที่กำหนดไว้ใน `.env.local`
- เราจะมาใส่ `NEXTAUTH_URL` หลังจาก Deploy สำเร็จ เพราะต้องใช้ URL ของ Vercel ที่ได้มาจากการ Deploy

$ .env.local

```
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET='Ey7nTKnggBc0bRN8WUjyShw2qzOZ6KW4fUyqcKBePxY='
```

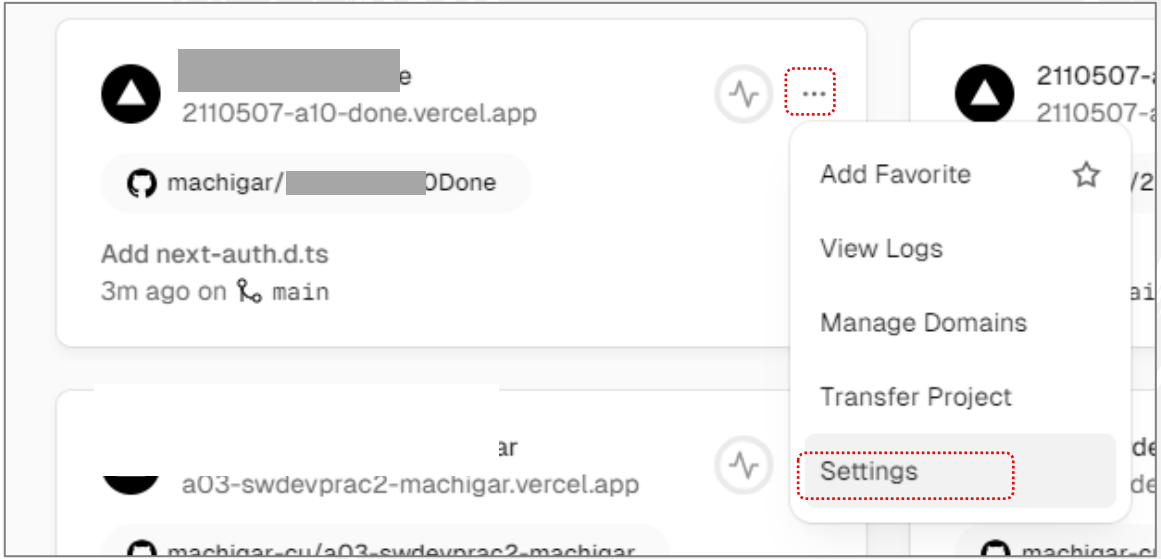


Deploy to Vercel

- Deploy เว็บไซต์ขึ้น Vercel จนเสร็จ และ add domain
- Copy domain URL ที่ได้

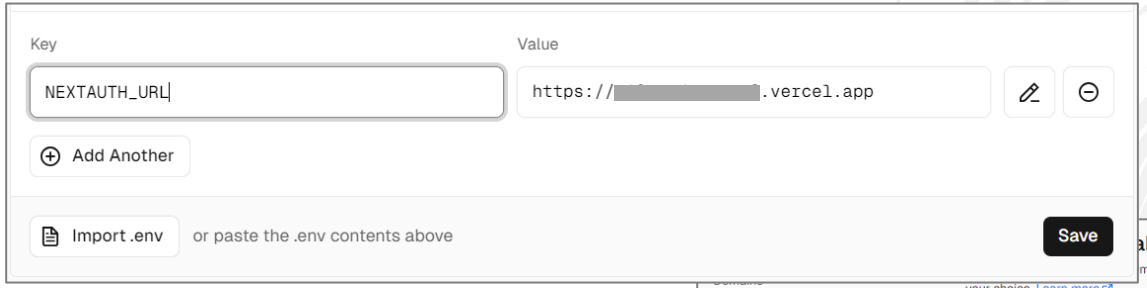
- ไปที่ Deployment ของเว็บไซต์
- เลือกปุ่มเมนู ... > Settings

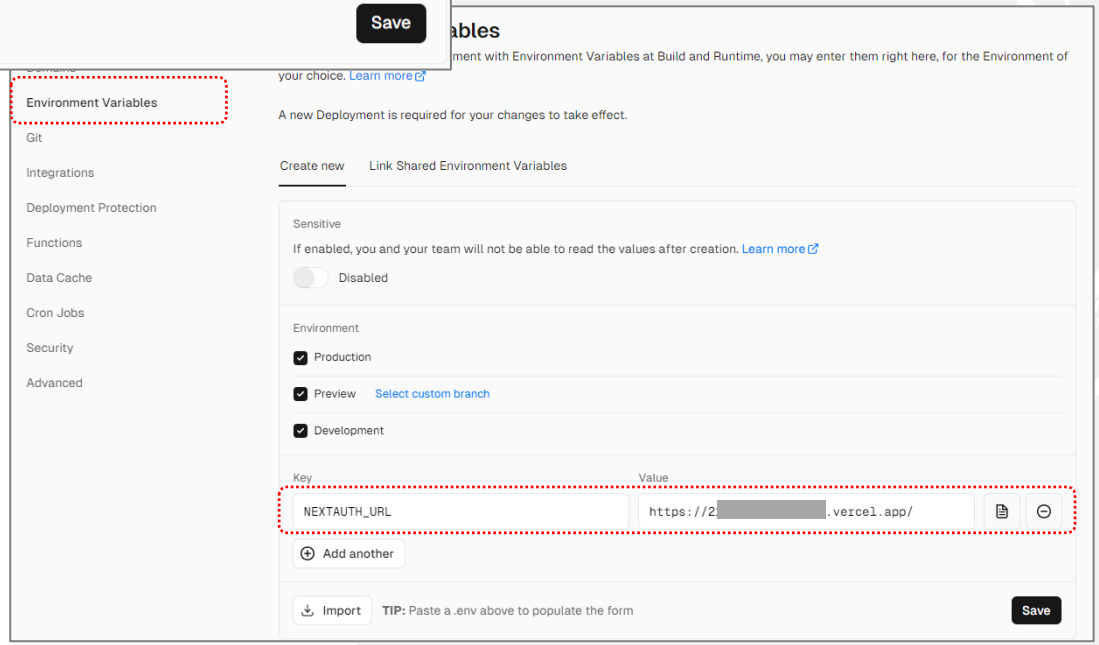
- เลือก Section Environment Variables

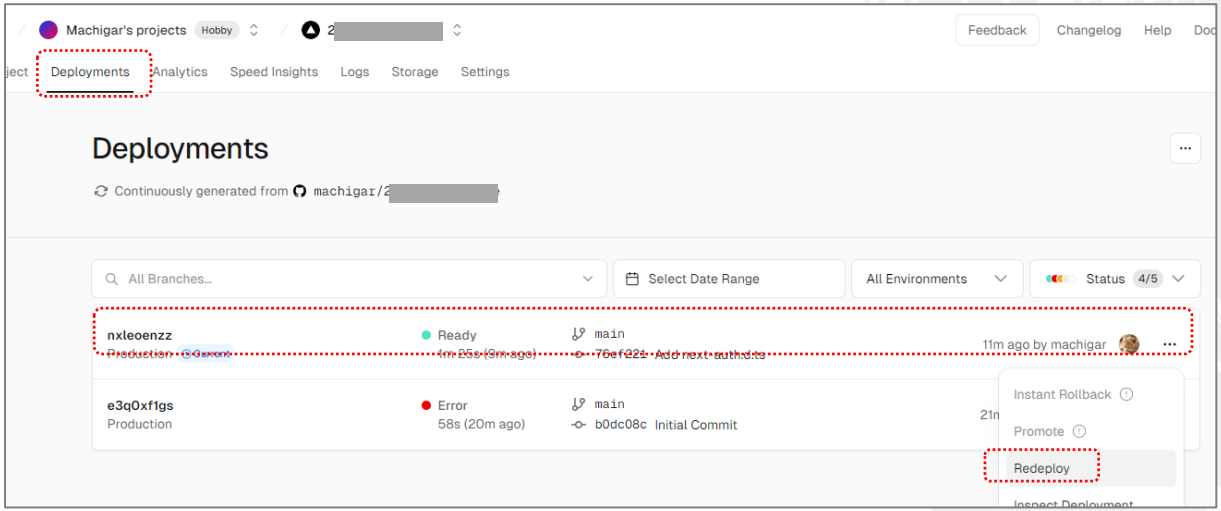
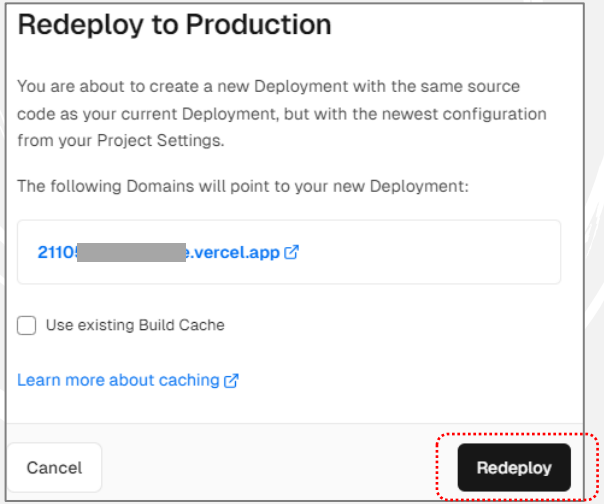


Deploy to Vercel

- เพิ่ม NEXTAUTH_URL กำหนดค่าเป็น ค่า vercel domain ที่ copy มา





Deploy to Vercel

- Redeploy Project
  - ไปที่ Deployments เลือก Deployment ล่าสุด
  - เลือก Redeploy และยืนยันการ Redeploy to Production